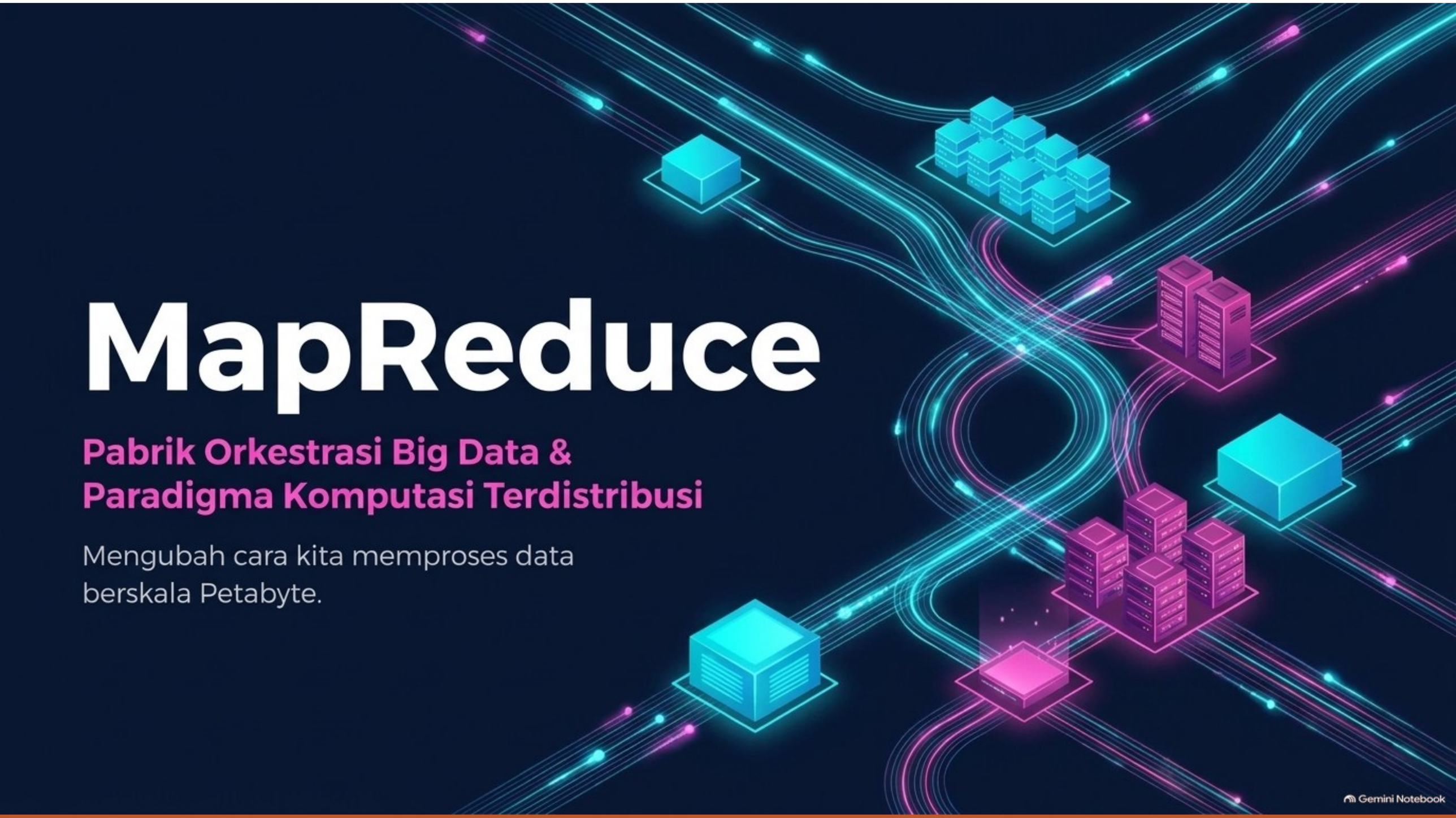

MapReduce
Pabrik Orkestrasi Big Data &
Paradigma Komputasi Terdistribusi
Mengubah cara kita memproses data
berskala Petabyte.
Gemini Notebook

Akar Masalah Big Data: Kemacetan Jaringan

Memindahkan data berukuran Terabyte untuk diproses oleh program adalah hal yang mustahil. Solusinya? Pindahkan programnya ke lokasi data.

Komputasi Tradisional (Sentralisasi)

MapReduce (Terdistribusi)



- **Fokus:** Membawa data ke kode (program).
- **Kelemahan:** Jaringan tersumbat (bottleneck) saat memindahkan data masif.
- **Skala:** Terbatas pada kapasitas satu server (Vertical Scaling).

- **Fokus:** Membawa kode (program) ke data (Data Locality).
- **Keunggulan:** Jaringan bebas hambatan, I/O lokal sangat cepat.
- **Skala:** Tak terbatas, menggunakan ribuan komputer biasa (Horizontal Scaling).

Gemini Notebook

Evolusi Menuju Pemrosesan Paralel

Bagaimana kebutuhan mesin pencari melahirkan standar industri Big Data.

1990-an: Ledakan Web.
Lahirnya internet memicu ledakan data digital tidak terstruktur (teks, gambar, log). Sistem database relasional mulai kewalahan.

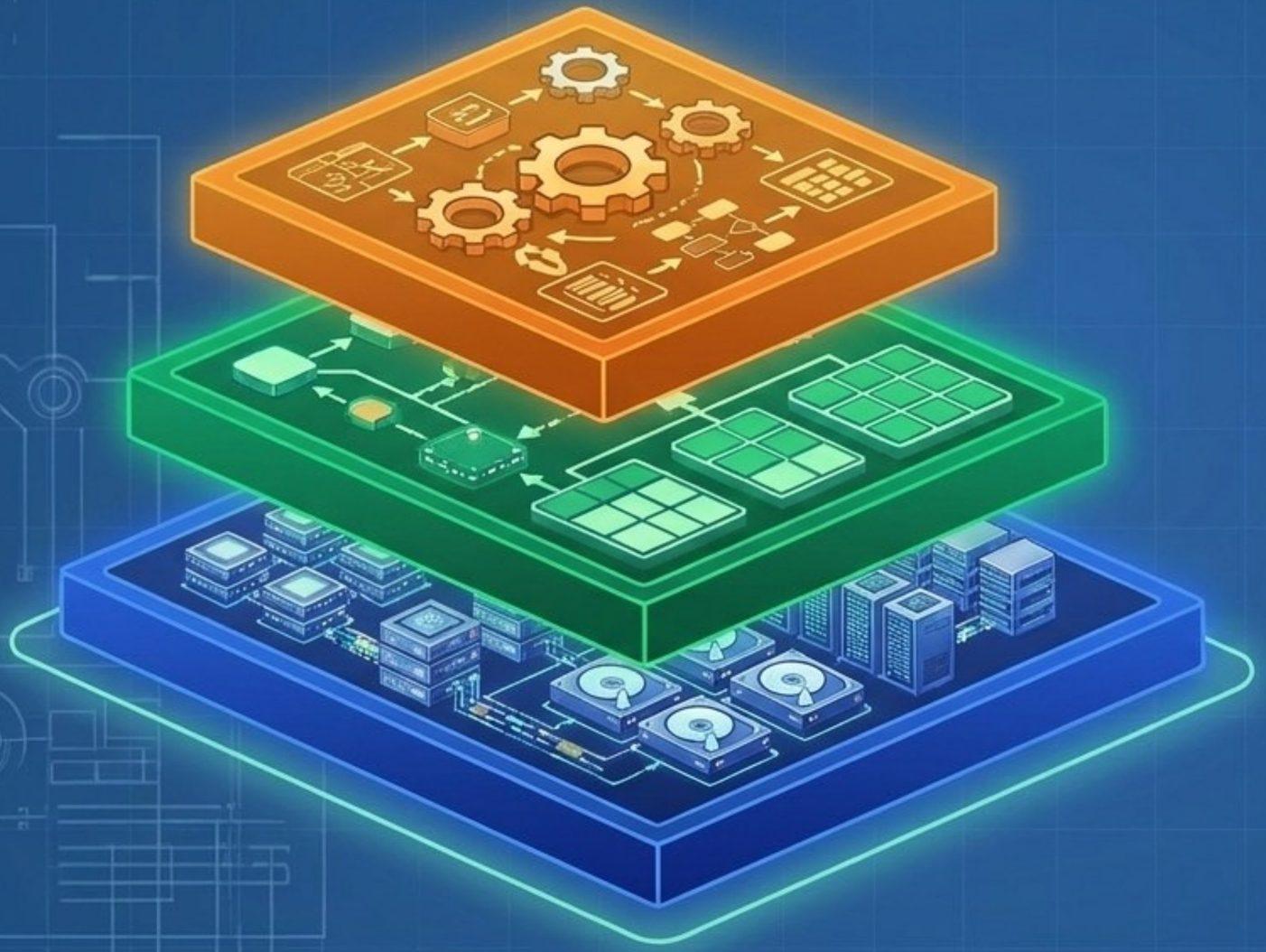
2004: Terobosan Google.
Google mempublikasikan whitepaper GFS (Google File System) dan MapReduce. Sebuah cetak biru untuk komputasi terdistribusi yang toleran terhadap kegagalan.



2006: Lahirnya Apache Hadoop.
Doug Cutting dan Mike Cafarella menciptakan Hadoop (dinamai dari Hadoop (dinamai dari mainan gajah). Menghadirkan implementasi open-source dari HDFS dan MapReduce yang kini digunakan oleh Facebook, Yahoo!, dan LinkedIn.

Gemini Notebook

Trinitas Ekosistem Hadoop



MapReduce (Komputasi)

Paradigma pemrosesan paralel yang mengeksekusi logika bisnis untuk mengubah data mentah menjadi wawasan.

YARN (Manajemen)

Sistem saraf yang memastikan alokasi RAM & CPU yang adil. Memberikan fleksibilitas penjadwalan multi-aplikasi.

HDFS (Penyimpanan)

Menjaga data skala petabyte tersimpan aman, terdistribusi, dan direplikasi untuk toleransi kegagalan.

Gemini Notebook

Fondasi Hadoop: Penyimpanan & Komputasi

MapReduce tidak bekerja sendiri. Ia sangat bergantung pada arsitektur penyimpanan dan manajemen sumber daya dari Hadoop.

Gudang Penyimpanan



HDFS (Hadoop Distributed File System)
- **Peran:** Gudang Penyimpanan.
- **Cara Kerja:** Memecah file raksasa menjadi Block berukuran tetap (default 128 MB atau 256 MB) dan mereplikasinya 3x di berbagai DataNode untuk fault tolerance.

Mandor Pabrik



YARN (Yet Another Resource Negotiator)
- **Peran:** Mandor Pabrik.
- **Cara Kerja:** Terdiri dari ResourceManager (mengatur jadwal & alokasi resource) dan NodeManager (mengawasi eksekusi tugas di tiap mesin).

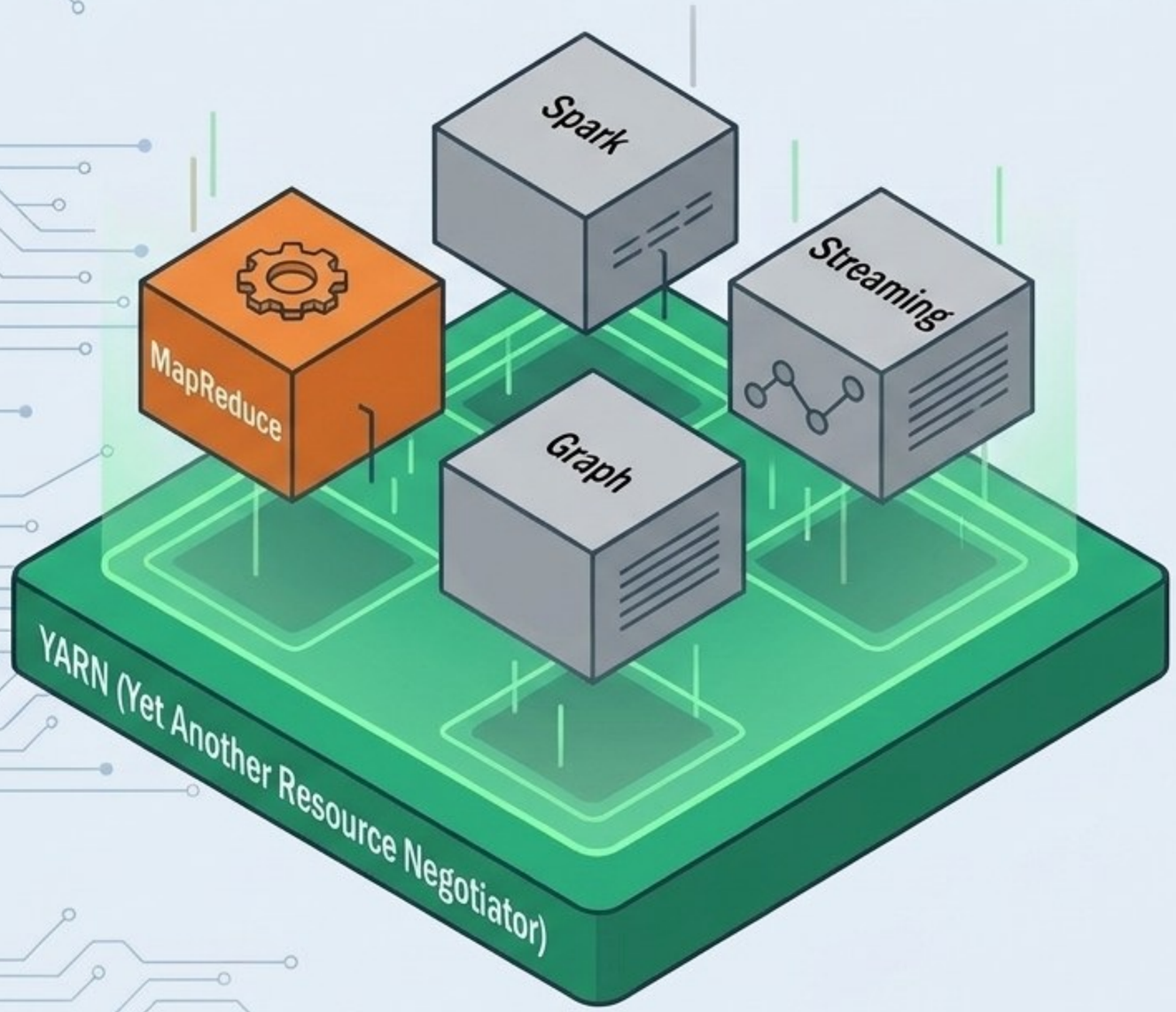
Lantai Pabrik



MapReduce
- **Peran:** Mesin Pemroses (Assembly Line).
- **Cara Kerja:** Memanfaatkan Data Locality untuk memproses data langsung di dalam DataNode tempat blok HDFS berada, mengurangi beban transfer data.

Gemini Notebook

YARN: Sistem Operasi Big Data Masa Kini

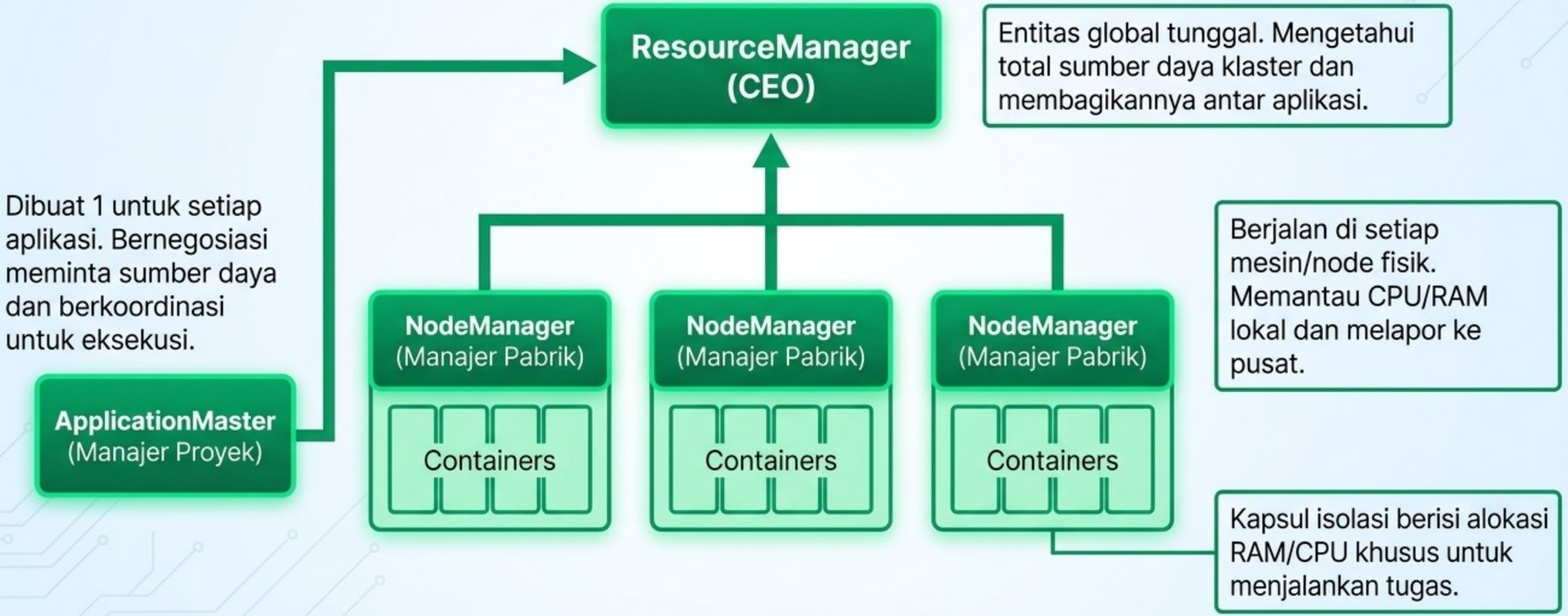


Sistem manajemen sumber daya **multi-tenant** yang memisahkan Manajemen Sumber Daya dari Penjadwalan Aplikasi.

Evolusi Platform: Hadoop tidak lagi eksklusif untuk batch processing MapReduce. YARN membuka jalan bagi berbagai arsitektur pemrosesan modern untuk berbagi klaster yang sama secara harmonis dan efisien.

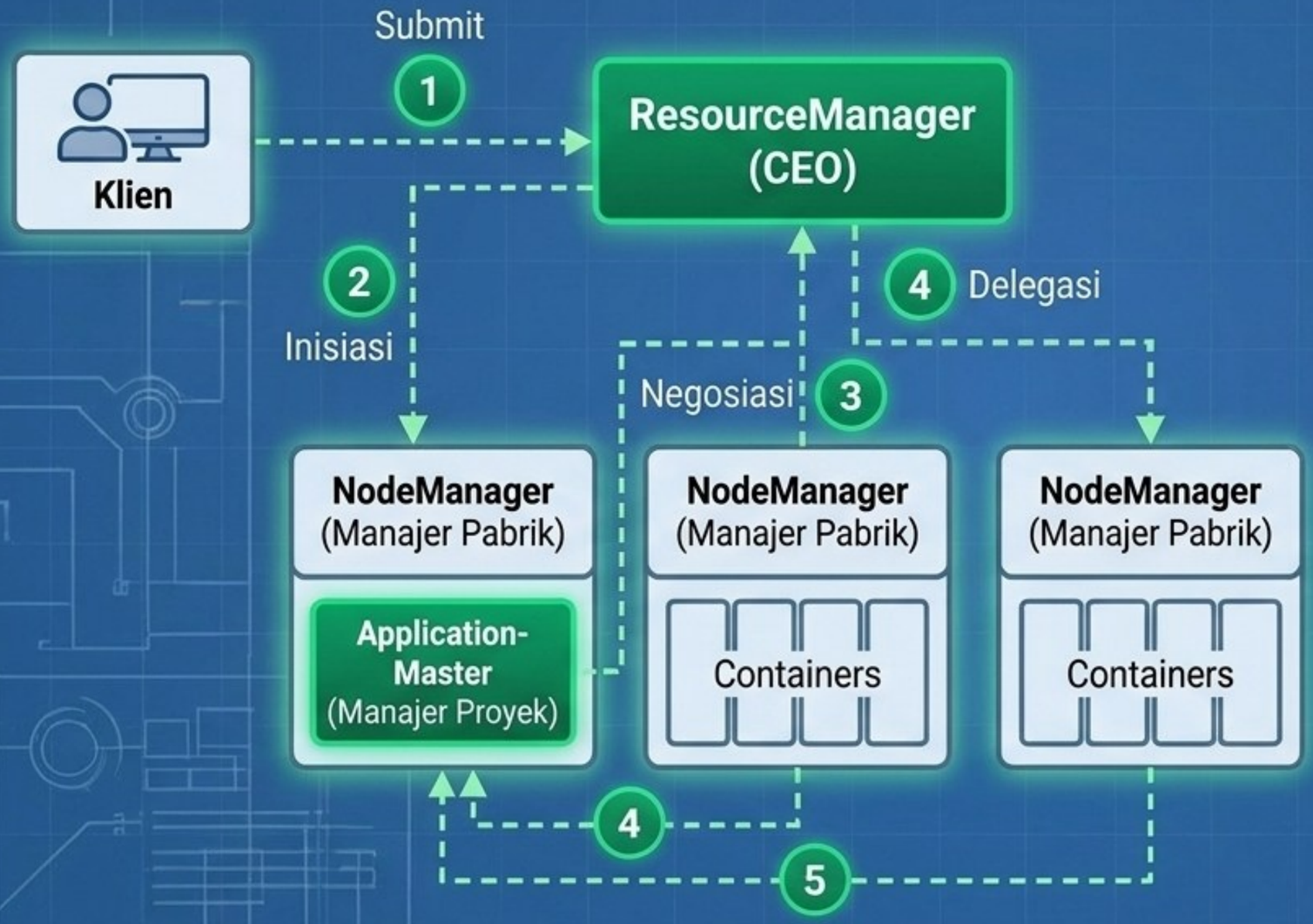
Gemini Notebook

Anatomi Manajerial YARN



Gemini Notebook

Koreografi Eksekusi Tugas

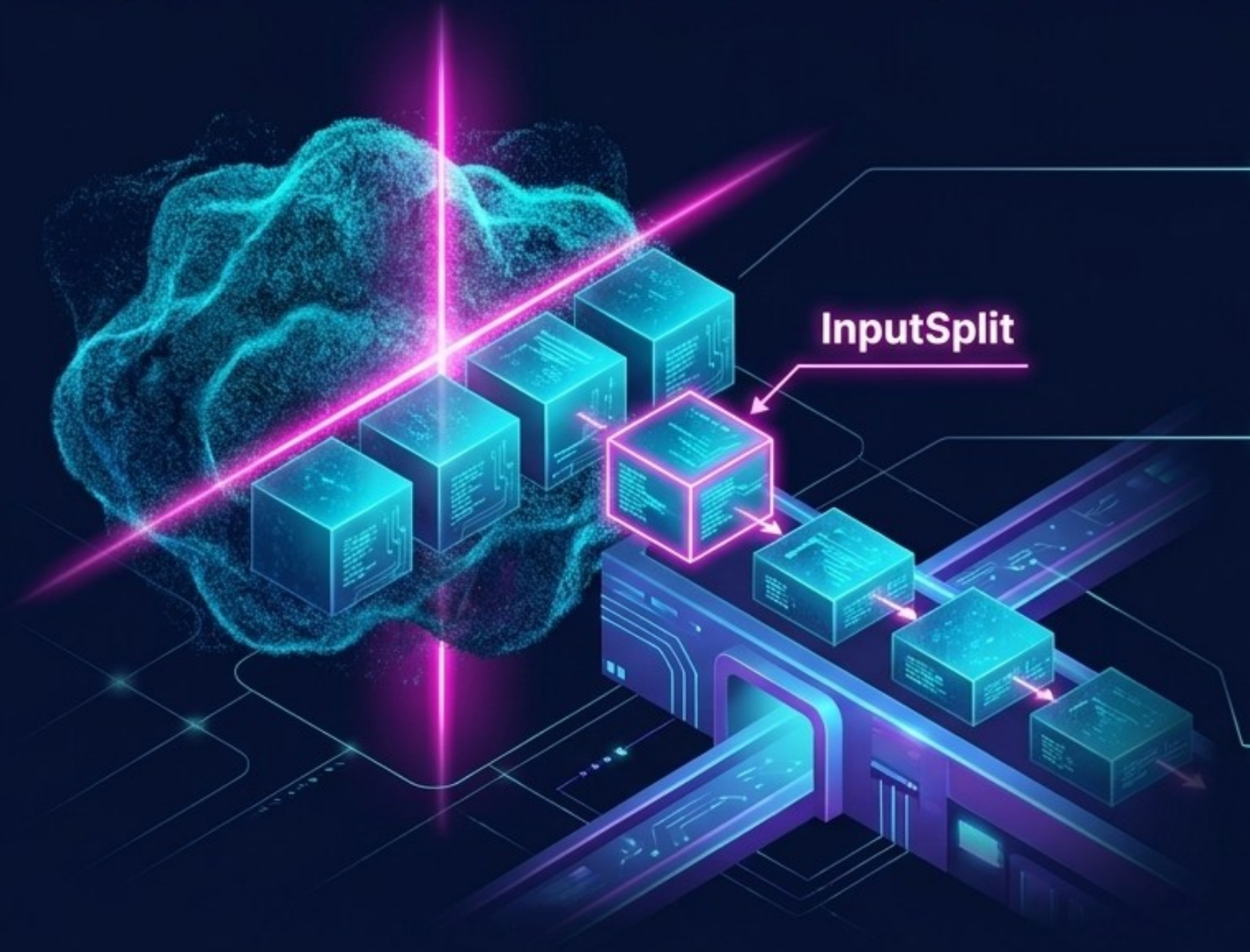


1. **Submit:** Klien mengirimkan aplikasi ke ResourceManager.
2. **Inisiasi:** ResourceManager memerintahkan NodeManager untuk memulai Container pertama (menampung ApplicationMaster).
3. **Negosiasi:** ApplicationMaster menghitung kebutuhan, lalu meminta tambahan alokasi Containers.
4. **Delegasi:** ApplicationMaster menyuruh NodeManager menjalankan pekerjaan komputasi di Containers baru.
5. **Eksekusi:** Pekerja berjalan dan melaporkan status kembali secara berkala.

Gemini Notebook

Perjalanan Data Fase 1: Input Split

Mengubah data mentah menjadi porsi yang dapat dikerjakan.



- **Pemecahan Data (Splitting):** Dataset masif tidak diproses sekaligus. Data dipecah menjadi bagian-bagian logis yang disebut **Input Split**.

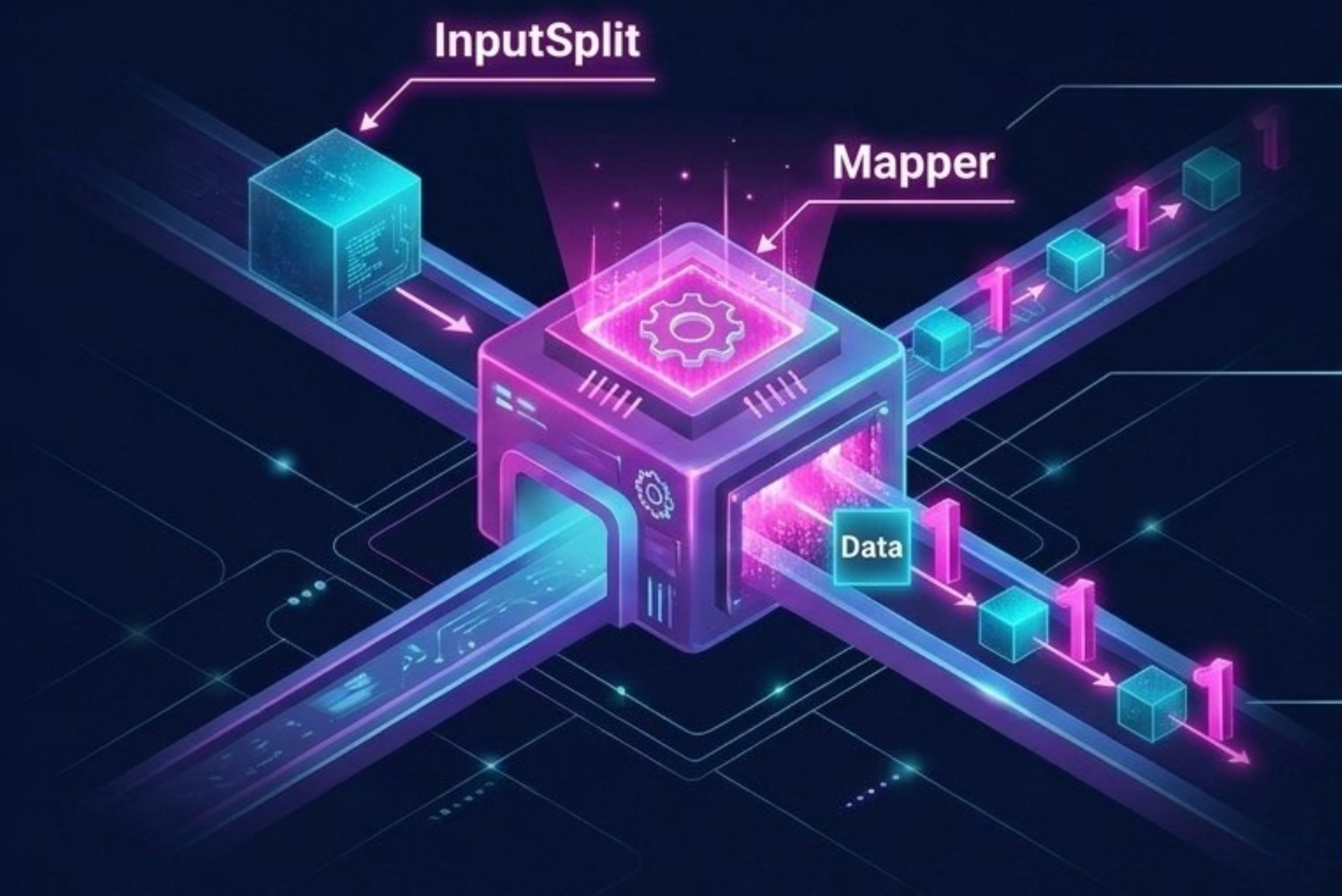
- **Keselarasan dengan HDFS:** Secara ideal, satu **Input Split** merepresentasikan tepat satu Block **HDFS** (misal 128 MB).

- **Penugasan (Assignment):** **Master Node** membagikan setiap **Input Split** kepada satu pekerja (**Worker Node / Mapper**). Jika memungkinkan, tugas diberikan kepada node yang secara fisik menyimpan blok data tersebut (Prinsip **Data Locality**).

Gemini Notebook

Perjalanan Data Fase 2: Map Phase

Eksekusi terisolasi dan ekstraksi informasi.



- **Eksekusi Paralel:** Setiap Input Split diproses secara independen dan bersamaan oleh fungsi Map di berbagai node di seluruh cluster. Tidak ada node yang menunggu node lain.

- **RecordReader:** Data mentah dari split dibaca sebaris demi sebaris dan diubah menjadi pasangan kunci-nilai (Key-Value) awal.

- **Fungsi Map (User-Defined):** Kode yang Anda tulis akan menerima input ini, melakukan filter atau transformasi, dan memancarkan (emit) Intermediate Key-Value Pairs baru. Contoh: Kata "Data" menjadi (Data, 1).

Gemini Notebook

Perjalanan Data Fase 3: Shuffle & Sort

Mengatur kekacauan. Membawa data yang sama ke tempat yang sama.



Fase Jembatan (Bridge Phase): Ini adalah fase yang diatur sepenuhnya oleh framework Hadoop, bukan oleh kode pengguna.

Partitioning (Pembagian): Sistem melihat kunci (key) dari setiap Intermediate Key-Value, dan menentukan Reducer mana yang akan menerimanya.

Shuffle (Pengacakan/Pemindahan): Data fisik dipindahkan melalui jaringan dari Mapper nodes ke Reducer nodes.

Sort & Group (Pengurutan): Data diurutkan berdasarkan kunci, dan semua nilai yang memiliki kunci yang sama digabungkan menjadi satu daftar (list).
Contoh: (Big, 1) dan (Big, 1) menjadi (Big, [1, 1]).

Gemini Notebook

Perjalanan Data Fase 4: Reduce & Output

Agregasi final dan penyimpanan hasil komputasi.

- **Input Terkelompok:** Fungsi Reduce menerima Key unik beserta seluruh daftar Values yang berasosiasi dengan kunci tersebut.

- **Fungsi Reduce (User-Defined):** Kode Anda memproses daftar nilai ini secara independen—biasanya melakukan operasi agregasi, perhitungan matematis, atau penggabungan string. Contoh: Menjumlahkan [1, 1] menjadi hasil akhir 2.



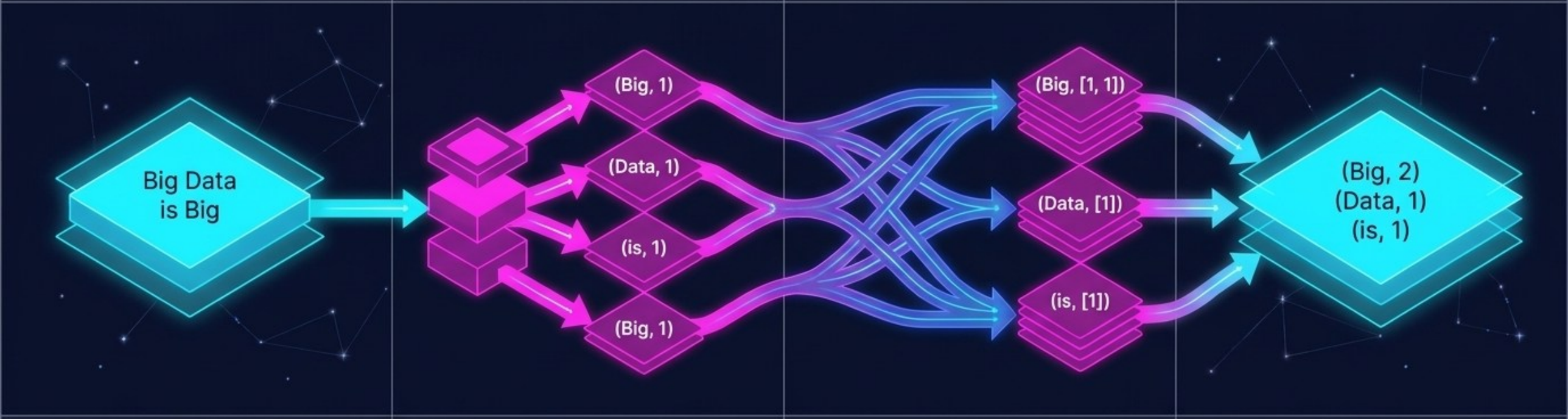
- **Final Output:** Hasil komputasi final (Key-Value akhir) ditulis kembali ke dalam HDFS. Hasil ini dapat digunakan sebagai laporan analisis atau menjadi input untuk tugas MapReduce selanjutnya.

Gemini Notebook

Ilustrasi Kasus: WordCount (Penghitung Kata)

Melacak perjalanan data teks dari pemecahan hingga agregasi akhir.

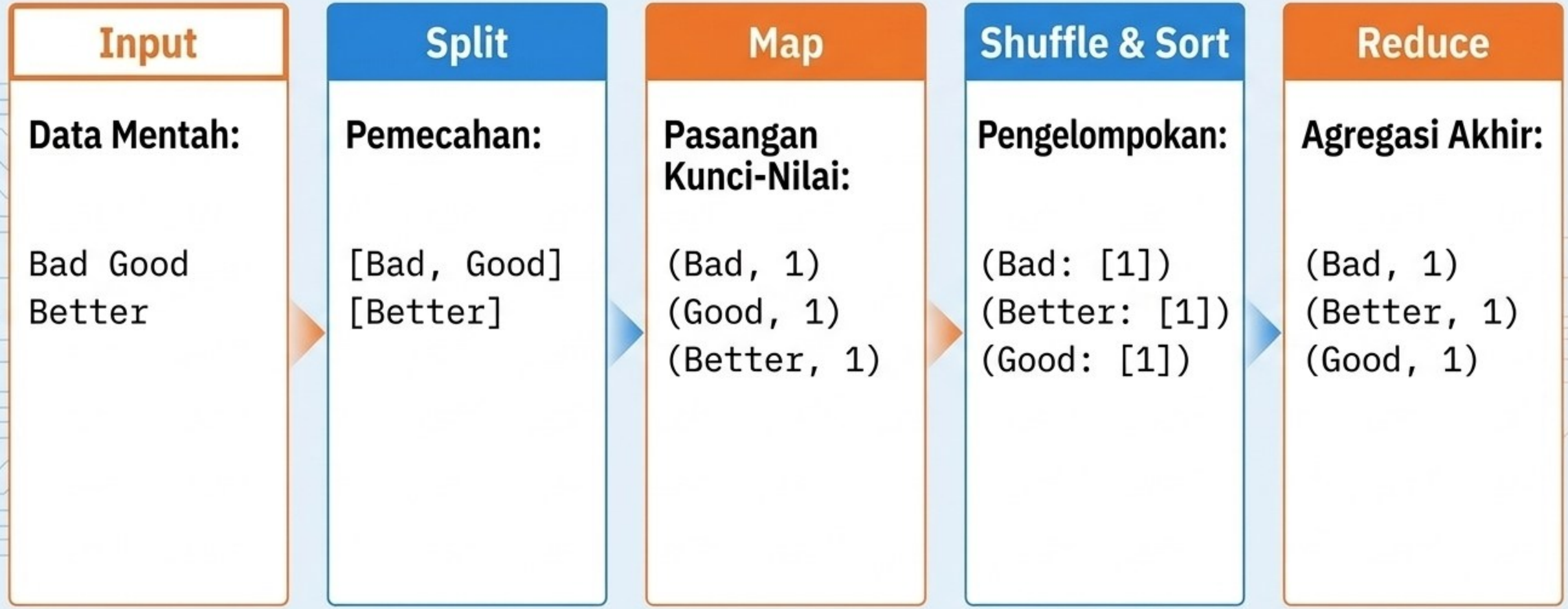
1. Input Split	2. Mapper	3. Shuffle/Sort	4. Reducer



1. Input Split: Dokumen teks diproses sebaris demi sebaris.	**2. Mapper:** Memecah kalimat menjadi kata (token), setiap kata dijadikan kunci, dan diberi nilai absolut 1.	**3. Shuffle/Sort:** Kerangka kerja (framework) secara otomatis mengelompokkan kata yang sama ke dalam satu array nilai.	**4. Reducer:** Menjumlahkan isi array untuk mendapatkan frekuensi total kemunculan setiap kata.

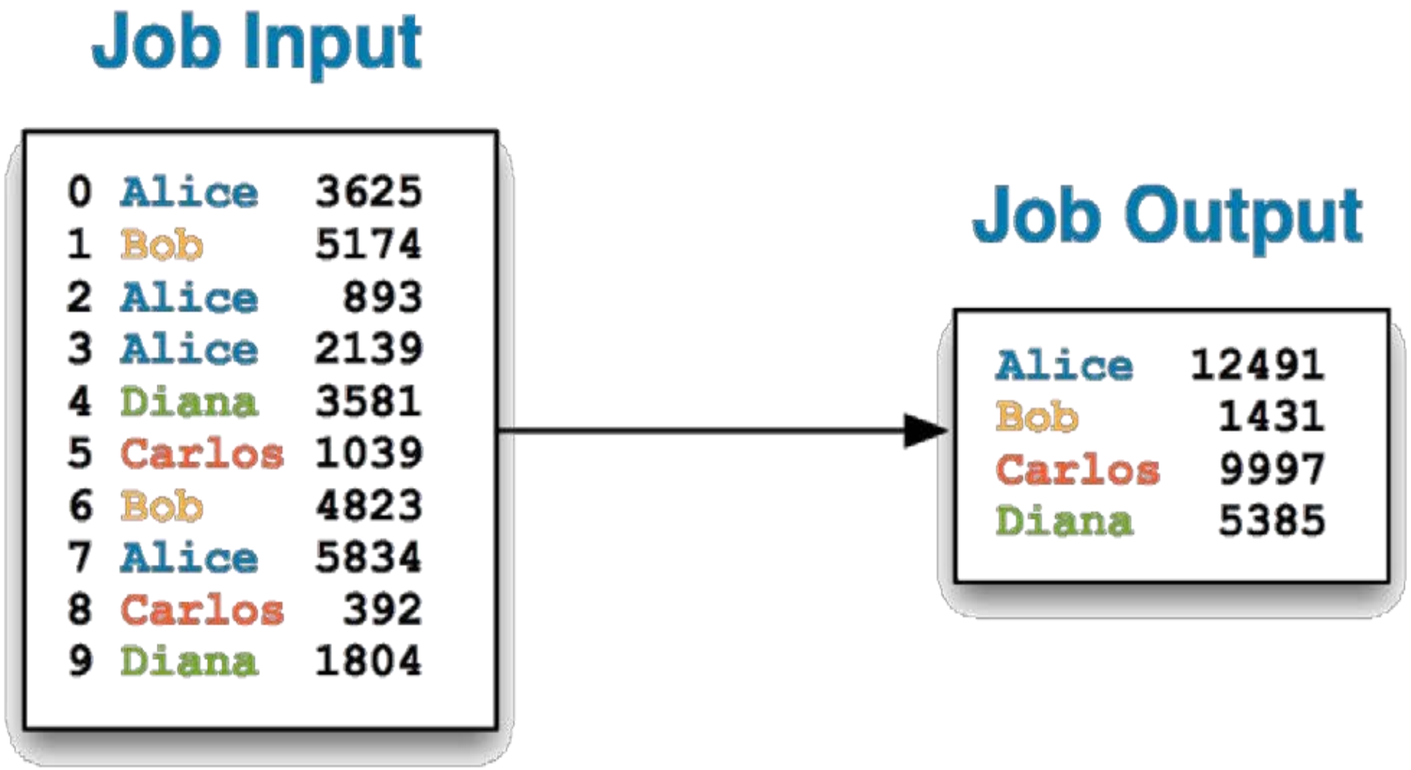
Gemini Notebook

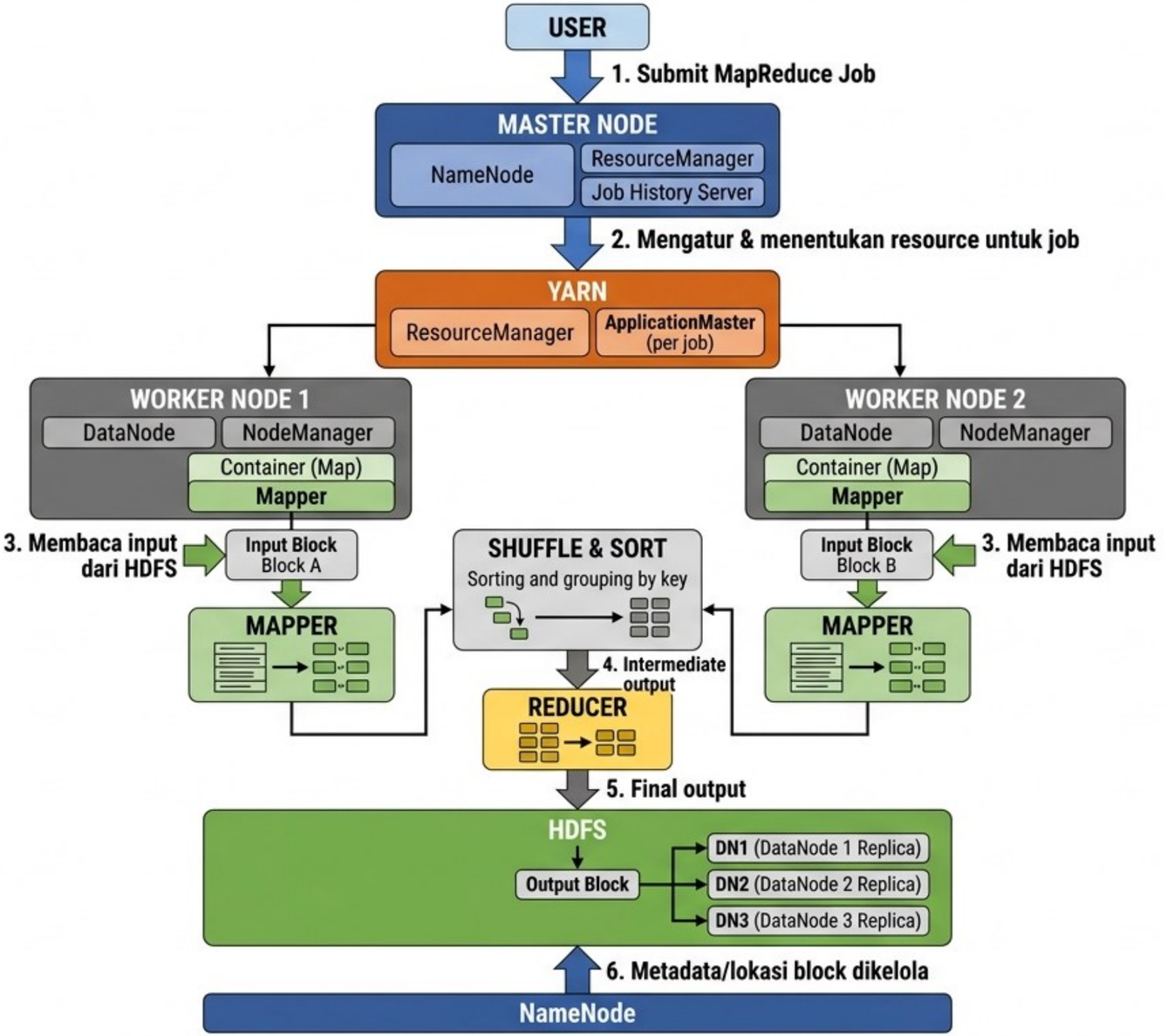
Simulasi Alur Kerja: Kasus WordCount



Gemini Notebook

- Contoh job pada MapReduce:
  - Input: file teks berisi order id, employee name, dan sale amount
  - Output: jumlah seluruh sales per employee




USER
1. Submit MapReduce Job
MASTER NODE
NameNode
ResourceManager
Job History Server
2. Mengatur & menentukan resource untuk job
YARN
ResourceManager
ApplicationMaster
(per job)
WORKER NODE 1
DataNode
NodeManager
Container (Map)
Mapper
WORKER NODE 2
DataNode
NodeManager
Container (Map)
Mapper
3. Membaca input
dari HDFS
Input Block
Block A
SHUFFLE & SORT
Sorting and grouping by key
Input Block
Block B
3. Membaca input
dari HDFS
MAPPER
MAPPER
4. Intermediate
output
REDUCER
5. Final output
HDFS
Output Block
DN1 (DataNode 1 Replica)
DN2 (DataNode 2 Replica)
DN3 (DataNode 3 Replica)
6. Metadata/lokasi block dikelola
NameNode

Pengenalan Pemrograman: Tipe Data Hadoop

Mengapa tidak menggunakan tipe data standar Java? Memperkenalkan antarmuka 'Writable'.

Tipe Asli Java	Tipe Hadoop Writable
`int`	`IntWritable`
`long`	`LongWritable`
`String`	`Text`
`double`	`DoubleWritable`
`float`	`FloatWritable`

Mengapa 'Writable'?

Dalam komputasi terdistribusi, data terus-menerus dikirim antar server (dikirim melalui jaringan). Tipe data standar Java tidak cukup optimal.

Tipe Writable dirancang khusus oleh Hadoop untuk serialization yang sangat cepat dan ringan, menghemat lebar pita (bandwidth) jaringan yang berharga.

Gemini Notebook

Cetak Biru Pemrogramanogramaman: Kelas Mapper

Mendefinisikan logika ekstraksi data.

Blueprint: The Code

```java
public class TokenizerMapper
    extends Mapper<Object, Text, Text, IntWritable>{

  private final static IntWritable one = new IntWritable(1);
  private Text word = new Text();

  public void map(Object key, Text value, Context context) {
     // Logika pemisahan string menjadi kata
     word.set(parsedWord);
     context.write(word, one);
  }
}
```

Visual Annotation



Input Key: Byte offset (posisi baris di file).

Input Value: Isi teks satu baris.

Context: Objek yang membawa data ke fase Shuffle & Sort.

Output Key: Kata yang diekstrak (Text)

Output Value: Angka statis 1 (IntWritable)

Gemini Notebook

Cetak Biru Pemrogramanogramaman: Kelas Reducer

Mendefinisikan logika agregasi akhir.

Blueprint: The Code

```
public class IntSumReducer
    extends Reducer<Text, IntWritable, Text, IntWritable> {

  private IntWritable result = new IntWritable();

  public void reduce(Text key, Iterable<IntWritable> values,
                     Context context) {
     int sum = 0;
     for (IntWritable val : values) {
        sum += val.get();
     }
     result.set(sum);
     context.write(key, result);
  }
}
```

Visual Annotation



Input Key: Kata spesifik (Text).

Input Values: Iterable list dari angka 1, misal: [1, 1, 1] (Iterable<IntWritable>).

Context: Objek yang menulis hasil akhir ke file sistem.

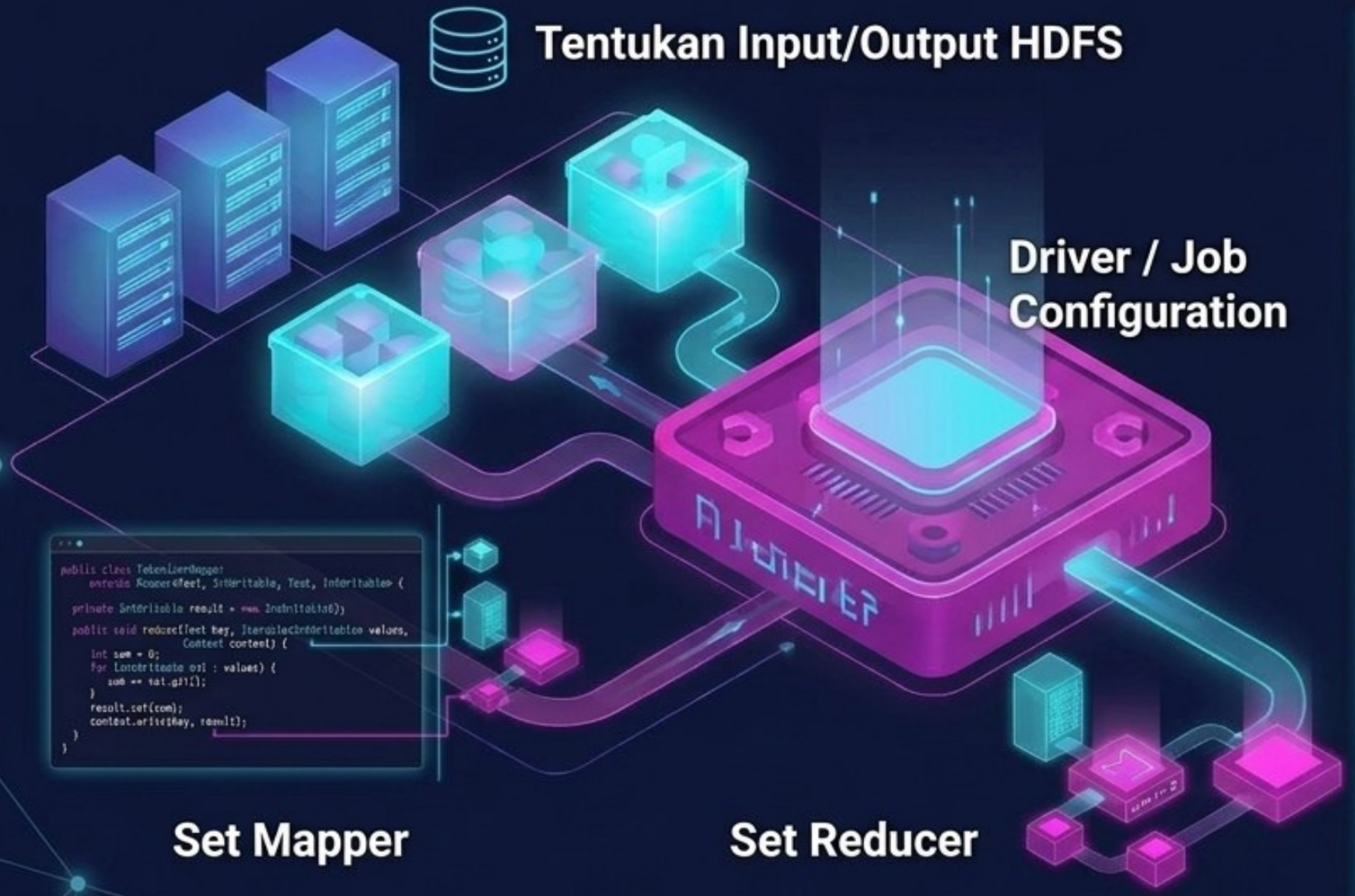
Output Key: Kata spesifik (sama).

Output Value: Jumlah total komputasi, misal: 3 (IntWritable).

Gemini Notebook

Sintesis: Driver & Job Configuration

Menyatukan HDFS, Mapper, dan Reducer ke dalam satu Orkestra.



- Sang Konduktor (The Orchestrator):

Kelas Driver adalah fungsi main() dari program MapReduce. Tanpa Driver, Mapper dan Reducer hanyalah cetak biru pasif.

- Tugas Konfigurasi (Job Setup):

- **Identifikasi Kode:** Memberi tahu YARN di mana kelas Mapper dan Reducer berada (job.setMapperClass, job.setReducerClass).
- **Definisi Tipe Data:** Memastikan sistem tahu tipe data apa yang diharapkan (Output Key/Value Class).
- **Pemetaan HDFS:** Mendefinisikan secara eksplisit path folder HDFS tempat data mentah diambil, dan path folder kosong tempat hasil final akan disimpan (FileInputFormat, FileOutputFormat).

Gemini Notebook

Kesimpulan: Filosofi di Balik Kode

Lebih dari sekadar fungsi pemrograman, MapReduce adalah cara pandang baru terhadap skala.

1. **Hukum Lokasi Data (Data Locality Law):** Pemrosesan skala petabyte hanya mungkin terjadi jika kode bergerak menuju data, bukan sebaliknya.
2. **Pemrosesan Terisolasi (Shared-Nothing):** Tugas bekerja secara independen, memungkinkan sistem bertahan hidup (*fault tolerance*) jika ada mesin yang mati mendadak.
3. **Kesederhanaan yang Skalabel:** Kerangka kerja menangani kekacauan jaringan, pemecahan data, dan pengurutan (*Shuffle & Sort*). Programmer hanya perlu fokus pada dua logika sederhana: Map dan Reduce.



"MapReduce menyembunyikan kompleksitas komputasi paralel di balik antarmuka fungsional yang sederhana, memungkinkan ribuan server bekerja seperti satu komputer utuh."

Gemini Notebook